

lab3_report

: 2022056562 서민용, 2023003227 이한주

1. Overall structure

```
module CPU(  
    input      clk,  
    input      rst,  
    output     halt  
);  
  
    // Split the instructions  
    // Instruction-related wires  
    wire [31:0] inst;  
    wire [5:0]  opcode;  
    wire [4:0]  rs;  
    wire [4:0]  rt;  
    wire [4:0]  rd;  
    wire [4:0]  shamt;  
    wire [5:0]  funct;  
    wire [15:0] immi;  
    wire [25:0] immj;  
  
    // Control-related wires  
    wire      RegDst;  
    wire      Jump;  
    wire      Branch;  
    wire      JR;  
    wire      MemRead;  
    wire      MemtoReg;  
    wire      MemWrite;  
    wire      ALUSrc;  
    wire      SignExtend;  
    wire      RegWrite;  
    wire [3:0] ALUOp;  
    wire      SavePC;  
  
    // Sign extend the immediate  
    wire [31:0] ext_imm;  
  
    // RF-related wires  
    wire [4:0]  rd_addr1;  
    wire [4:0]  rd_addr2;  
    wire [31:0] rd_data1;  
    wire [31:0] rd_data2;
```

```
// CPU.v cont'...  
// MEM-related wires  
    wire [31:0] mem_addr;  
    wire [31:0] mem_write_dat  
a;  
    wire [31:0] mem_read_dat  
a;  
  
    // ALU-related wires  
    wire [31:0] operand1;  
    wire [31:0] operand2;  
    wire [31:0] alu_result;  
  
    // Define PC  
    reg [31:0] PC;  
    reg [31:0] PC_next;  
  
    // Define the wires  
  
    assign halt = (in  
st == 32'b0);  
  
    always @(*) begin  
        // ...TODO  
    end  
  
    // Update the Clock  
    always @(posedge clk) begin  
        if (rst)    PC <= 0;  
        else begin  
            PC <= PC_next;  
        end  
    end  
  
endmodule
```

```
module MEM(  
    input      clk,
```

```

reg [4:0]      wr_addr;
reg [31:0]     wr_data;

```

```

module CTRL(
    // input opcode and funct
    input [5:0] opcode,
    input [5:0] funct,

    // output various ports
    output reg RegDst,
    output reg Jump,
    output reg Branch,
    output reg JR,
    output reg MemRead,
    output reg MemtoReg,
    output reg MemWrite,
    output reg ALUSrc,
    output reg SignExtend,
    output reg RegWrite,
    output reg [3:0] ALUOp,
    output reg SavePC
);

    always @(*) begin
        // FIXME
    end
endmodule

```

```

input          rst,

input [31:0]    inst_addr,
r,
output reg [31:0] inst,

input [31:0]    mem_addr,
input          MemWrite,
input [31:0]    mem_write_data,
output reg [31:0] mem_read_data
);

reg [31:0] memory [0:8191];

initial begin
    $readmemh("initial_mem.mem", memory);
end

// Read instruction + memory data
always @(*) begin
    inst = memory[(inst_addr >> 2)];
    mem_read_data = memory[(mem_addr >> 2)];
end

always @(posedge clk) begin
    if (!rst)
        if (MemWrite)
            memory[(mem_addr >> 2)] <= mem_write_data;
end

endmodule

```

- 기존 주어진 스켈레톤 코드의 형태입니다.
 - RF.v와 ALU.v 는 HW1,2 의 코드와 같습니다.
- 전체적인 구조에서 하나 알아가야 할 점은 CPU.v 와 MEM.v 만 clk가 input으로 들어간다는 점입니다.
- 나머지 모듈에서는 clk의 변화에 따라서 CPU.v와 MEM.v에서 변화되는 변수들에 대한 변화를 감지하면서 연산이 되어야겠습니다. posedge clk 마다
 - CPU.v에서는 PC의 값만 PC_next의 값으로 업데이트 시켜주고 있고,

- 구현해야할 부분 역시 PC값이 변화가 되었다, 즉 다음 inst를 읽을 차례일 때 해주어야할 동작들을 작성해주면 되겠습니다.
- MEM.v에서는 MemWrite가 1이라면 mem_write_data의 값을 memory[(mem_addr >> 2)]로 넣어주고 있습니다.
 - memory 값의 변화를 감지하여 inst와 mem_read_data의 값을 업데이트 시키고 있기 때문에,
 - posedge clk → memory, (MemWrite == 1 일 때만) inst, mem_read_data 의 값이 함께 변합니다.
 - 또한 MEM.v 의 input인 inst_addr와 mem_addr의 변화를 감지하기도 하기 때문에, 이 두 input을 CPU.v에서 연결 시켜주는 wire 역시 신경써야합니다.
- 수업에서 제공되었던 single cycle CPU의 그림에서와 달리, CTRL 모듈에 추가적으로 3개의 control signal이 있습니다.
 1. SignExtend : 각 inst 별로 immediate 에 대해서 signed로 취급할 지, unsigned(디폴트)로 취급할 지에 대해 다른 extend를 사용해야하기 때문에 존재합니다.

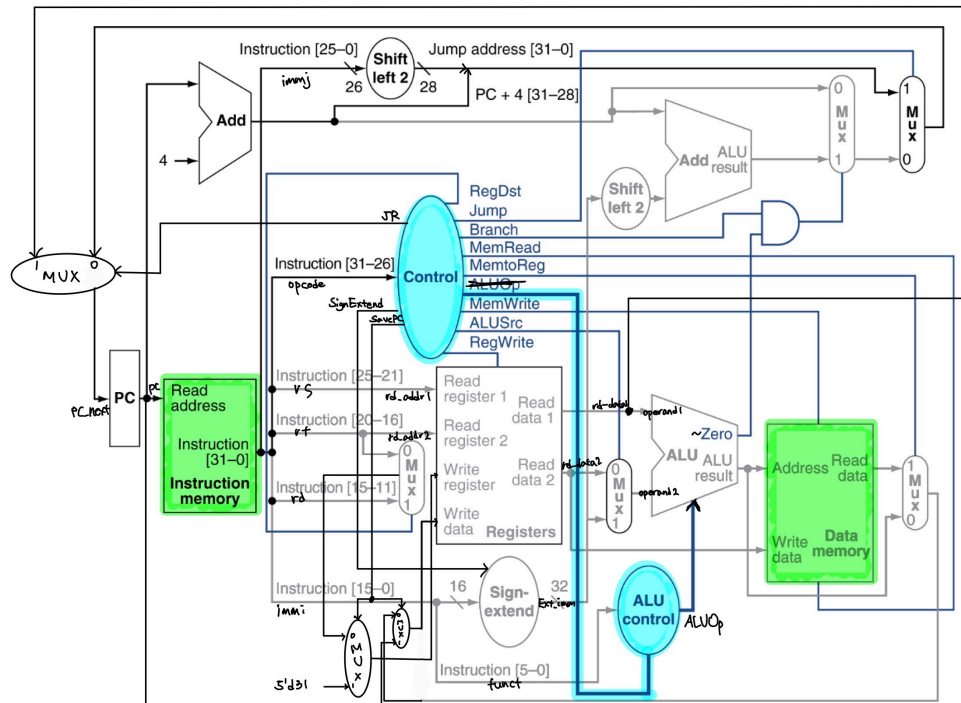
ADDIU rt, rs, imm	rt = rs + sign(imm)
AND rd, rs, rt	rd = rs & rt
ANDI rt, rs, imm	rt = rs & zero(imm)

2. JR : 기존 수업에선 생략 했던 jr inst에 대해서 다루어주어야하기 때문에 존재합니다.
3. SavePC : 기존 수업에선 생략했던 jal inst에 대해서, function call 이후 돌아가야할 PC 위치를 r31 register에 저장해야하기 때문에 존재합니다.

JAL target	r31 = pc; pc=pc_upper (target << 2)
------------	--

- 또한 구현 이후 wire로 각 모듈에 필요한 값을 연결해주면 되겠습니다.

jal & jr inst를 포함한 CPU Microarchitecture :



- 제공된 스켈레톤 코드를 고려하여 다음과 같이 설계하였습니다.
 - jr에 대해서는, GPR[rs]의 값, 즉 rd_data1이 PC값이 되어야하기 때문에, JR signal을 input으로 하는 MUX에 의해서 1이라면 rd_data1의 값을 PC_next로 설정합니다.

R-type Instructions – Control ver.

0	rs	rt	rd	shamt	funct
6-bit	5-bit	5-bit	5-bit	5-bit	6-bit

R-type

- There exists an R-type control instruction
 - jr
- Generally used along with jal label (upon a function call)
- Assembler and semantics
 - Corresponding assembly: **jr rs**
 - PC = GPR[rs]
- jal에 대해서는, target address는 기존 jump의 target과 같기 때문에 이를 활용하고, GPR[ra], 즉 31번째 register인 r31에 저장 PC+4의 값을 저장합니다.
- 이는 r31이라는 special purposed register로 고정되어 있기 때문에 SavePC을 input으로 하는 MUX에 의해서 1이라면 5'd31로 설정합니다.
- Corresponding assembly: **jal label**
 - GPR[ra] = PC + 4
 - save the next PC to ra (a dedicated register)
 - PC = target

- 추후 언급될 예정이지만, IMEM과 DMEM이 하나의 unit으로 취급되고, Control 과 ALU control이 하나의 unit으로 취급되어야하고, 이는 MEM.v, CTRL.v에 반영됩니다.

2. Implementation

2-1. CPU.v

```
// CPU.v
// ...
// Define the wires
assign opcode = inst[31:26];
assign rs = inst[25:21];
assign rt = inst[20:16];
assign rd = inst[15:11];
assign shamt = inst[10:6];
assign funct = inst[5:0];

assign immi = inst[15:0];
assign immj = inst[25:0];

assign rd_addr1 = rs;
assign rd_addr2 = rt;

assign operand1 = rd_data1;

assign mem_addr = alu_result;
assign mem_write_data = rd_data2;

assign ext_imm = (SignExtend) ? {{16{immi[15]}}, immi}: immi;

assign operand2 = ALUSrc ? ext_imm : rd_data2;

assign halt = (inst == 32'b0);

always @(*) begin
    if (SavePC) begin
        wr_addr = 5'd31;
        wr_data = PC+4;
    end
    else begin
        if (RegDst) wr_addr = rd;
        else wr_addr = rt;
        if (MemtoReg) wr_data = mem_read_data;
        else wr_data = alu_result;
    end
end
```

```

        if (JR) PC_next = rd_data1;
    else begin
        // according to MIPS assembly.
        if (Jump) PC_next = ((PC+4) & 32'hF0000000) | (immj << 2);
        // according to assignment figure.
        else begin
            // according to MIPS assembly.
            // according to assignment figure.
            if (|alu_result && Branch) PC_next = (PC + 4) + (ext_imm <
< 2);

            else PC_next = PC + 4;
        end
    end
end
end

// ...
MEM mem (
    .clk(clk),
    .rst(rst),
    .inst_addr(PC),
    .inst(inst),
    .mem_addr(mem_addr),
    .MemWrite(MemWrite),
    .mem_write_data(mem_write_data),
    .mem_read_data(mem_read_data)
);

```

- 매 posedge clk 가 바뀔 때마다 PC의 값이 바뀌고, 이 PC의 값이 MEM.v의 inst_addr로 연결되고, inst_addr의 변화에 따라서 MEM.v 에서 instruction memory에 대한 동작이 이루어집니다. 즉, inst가 fetch가 되게 됩니다.
- fetch 된 inst는 곧바로 assign을 통해서 rs, rt 등으로 연결되어 decode 되고,
- 각 wire들이 assign을 통해서 rd_addr1, rd_addr2 등으로 연결되어서, 이 값은 RF.v 로 연결되어서 Register에서 필요한 값을 읽게 됩니다.
- 그 이후 알맞는 ALU 연산이 이루어집니다. alu_result는 mem_addr 과 assign을 통해 연결되어 있는데,
- 이를 통해 MEM.v 에서 이 주소값에 따라 memory에 있는 데이터를 mem_read_data에 값을 받아오게 됩니다.
 - 이 값은 만일 해당 cycle에서 MemtoReg의 값이 1이라면, wr_data에 값이 담기게 됩니다.
 - 1이 아니라면, alu_result의 값이 wr_data에 담기게 됩니다.
- 이해를 편하게 하기 위해서 선형적으로 과정을 나열 했지만, 모든 과정이 1 Cycle 내에 이루어짐을 인지해야 합니다.
- 또한, Single cycle CPU이기 때문에, MemRead control 신호는 아직 하는 일이 없습니다.
 - 이는 초반 overall structure에서 제시되었듯, IMEM과 DMEM이 하나의 unit으로써 취급이 되는데, 매 Cycle마다 IMEM에서 inst를 항상 fetch 해오기 때문에, Memory에서 필요한 데이터를 read 하는가를 알리는 MemRead는 의미가 없다는 것입니다.

◦ 이는 Multicycle CPU가 되었을 때 의미가 있어질 값이 되겠습니다.

- 사실 교수님께서 제공해주신 MEM.v에 MemRead를 input으로 받지 않아서 그렇고, 추후엔 아마 MEM.v에 input으로 들어가게 되고 다음 두 줄이 각각 다른 always 문으로 분리가 될 듯 합니다.

```
always @(*) begin
    inst = memory[(inst_addr >> 2)];
    mem_read_data = memory[(mem_addr >> 2)];
end
```

- 또한 Jump, JR, SavePC, MemtoReg, RegDst 와 같은 control signal에 대해서는 if 문으로, ALUSrc, SignExtend에 대해서는 삼항연산자를 통해 분기를 나누고 있는데,
- 이는 MUX라는 개별 unit이 없기 때문에 TOP module의 역할을 하는 CPU.v에서 처리해주었습니다.

2-2. CTRL.v

```
// CTRL.v
// ...
always @(*) begin
    RegDst = 0;
    Jump = 0;
    Branch = 0;
    JR = 0;

    MemRead = 0;
    MemtoReg = 0;
    MemWrite = 0;
    ALUSrc = 0;
    SignExtend = 0;
    RegWrite = 0;
    SavePC = 0;
    if(opcode == `OP_RTYPE) b
    egin
        RegDst = 1;
        RegWrite = 1;
        if(funcnt == `FUNCT_J
        R) begin
            JR = 1;
            RegWrite = 0;
        end
        else begin
            case(funcnt)
                `FUNCT_ADDU:
ALUOp = `ALU_ADDU;
                `FUNCT_AND:
ALUOp = `ALU_AND;
```

```
// CTRL.v cont'...
`OP_ADDIU: begin
    RegWrite = 1;
    ALUSrc = 1;
    SignExtend = 1;
    ALUOp = `ALU_ADD
U;
    end

    `OP_SLTI: begin
        RegWrite = 1;
        ALUSrc = 1;
        SignExtend = 1;
        ALUOp = `ALU_SLT;
    end

    `OP_SLTIU: begin
        RegWrite = 1;
        ALUSrc = 1;
        SignExtend = 1;
        ALUOp = `ALU_SLT
U;
```

```

        `FUNCT_NOR:
        ALUOp = `ALU_NOR;
        `FUNCT_OR:  A
        LUOp = `ALU_OR;
        `FUNCT_SLT: A
        LUOp = `ALU_SLT;
        `FUNCT_SLTU:
        ALUOp = `ALU_SLTU;
        `FUNCT_SUBU:
        ALUOp = `ALU_SUBU;
        `FUNCT_XOR:
        ALUOp = `ALU_XOR;
        `FUNCT_SLL:
        ALUOp = `ALU_SLL;
        `FUNCT_SRA:
        ALUOp = `ALU_SRA;
        `FUNCT_SRL:
        ALUOp = `ALU_SRL;
    endcase
end
end
case(opcode)
    `OP_J: begin
        Jump =
1;
    end

    `OP_JAL: begin
        Jump =
1;

        RegWrite =
1;

        SavePC =
1;
    end

    `OP_BEQ: begin
        Branch =
1;

        SignExtend =
1;

        ALUOp = `ALU_EQ;
    end

    `OP_BNE: begin
        Branch =
1;

        SignExtend =

```

```

end

    `OP_ANDI: begin
        RegWrite =
1;

        ALUSrc =
1;

        ALUOp = `ALU_AND;
    end

    `OP_ORI: begin
        RegWrite =
1;

        ALUSrc =
1;

        ALUOp = `ALU_OR;
    end

    `OP_XORI: begin
        RegWrite =
1;

        ALUSrc =
1;

        ALUOp = `ALU_XOR;
    end

    `OP_LUI: begin
        RegWrite =
1;

        ALUSrc =
1;

        ALUOp = `ALU_LUI;
    end

    `OP_LW: begin
        MemRead =
1;

        MemtoReg =
1;

        ALUSrc =
1;

        SignExtend =
1;

        RegWrite =
1;

        ALUOp = `ALU_ADD
    end
end

```



```
1;

        ALUOp = `ALU_NEQ;
    end
```

```
        `OP_SW: begin
            MemWrite =

1;

            ALUSrc =

1;

            SignExtend =

1;

            ALUOp = `ALU_ADD

U;

        end
    endcase
end
```

- CPU.v 는 간단하게 각 opcode와 funct 값에 따라서 각 Signal을 알맞게 넣어주면 되겠습니다.
- 수업시간에 다루었던 control logic 에 대한 slide를 많이 참고하였습니다.

	When De-asserted	When asserted	Equation
RegDest	GPR write select according to <i>rt</i> , i.e., inst[20:16]	GPR write select according to <i>rd</i> , i.e., inst[15:11]	<i>opcode</i> ==0
ALUSrc	2 nd ALU input from 2 nd GPR read port	2 nd ALU input from sign-extended 16-bit immediate	(<i>opcode</i> !=0) && (<i>opcode</i> !=BEQ) && (<i>opcode</i> !=BNE)
MemtoReg	Steer ALU result to GPR write port	steer memory load to GPR wr. port	<i>opcode</i> ==LW
RegWrite	GPR write disabled	GPR write enabled	(<i>opcode</i> !=SW) && (<i>opcode</i> !=Bxx) && (<i>opcode</i> !=J) && (<i>opcode</i> !=JR))

	When De-asserted	When asserted	Equation
MemRead	Memory read disabled	Memory read port return load value	<i>opcode</i> ==LW
MemWrite	Memory write disabled	Memory write enabled	<i>opcode</i> ==SW
Jump	According to Branch	next PC is based on 26-bit immediate jump target	(<i>opcode</i> ==J) (<i>opcode</i> ==JAL)
Branch	next PC = PC + 4	next PC is based on 16-bit immediate branch target	(<i>opcode</i> ==Bxx) && "bcond is satisfied"

- 또한 케이스를 나눌 때 R-type inst 인가가 가장 큰 분기문이 되고, 이후 slide처럼 if문을 활용하여 opcode를 따져주지 않고 case 문을 쓴 이유는 비직관적이기도 하고 추후 Multicycle CPU 시, case 문을 활용하는 게 코드 작성에 더 유리할 것 같다고 생각했기 때문입니다.
- 또한 초반 overall structure에서 제시 되었듯, 제공된 스켈레톤 코드에서는 Control unit 과 ALU control unit이 하나로써 취급되기 때문에, CTRL.v 내에서 ALUOp 값을 일일이 따져줍니다.

3. result

- 제공된 모든 testcase에 대해서 모두 통과한 모습입니다.
- 각 testcase 별로 실행시간이 달라 모든 스크린샷을 첨부했습니다.

```
minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
ERROR: RF.v:35: $readmemh: Unable to open initial_reg.mem for reading.
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 6225000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %
```

testcase1

```
minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 1985000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %
```

testcase2

```

minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 1655000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %

```

testcase3

```

minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 6815000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %

```

testcase4

```

minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 3165000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %

```

testcase5

```

minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 2005000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %

```

testcase6

```

minyongseo@Minui-MacBookAir HW03_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:59: $finish called at 5085000 (1ps)
minyongseo@Minui-MacBookAir HW03_HanJoo %

```

testcase7

4. Troubleshooting

5. Further Question

- Jump 명령어에서는 offset 범위의 한계로 인해 실제 이동 가능한 범위가 26비트밖에 되지 않고, 하위 두 개의 비트는 0으로, 상위 네 개의 비트는 이전 PC의 상위 4비트를 가져오게 됩니다. 그런데, MIPS ISA의 명세와 과제 명세 내의 diagram에서 사소한 차이점이 있었습니다. 상위 4비트의 값을 PC에서 가져오는지, PC+4에서 가져오는지에 따라서 Jump의 위치가 완전히 달라질 수 있는 문제점을 발견했습니다.
- 예를 들어, 현재 PC가 0x0FFFFFFC이고, 해당 지점의 명령어가 Jump라면 상황에 따라 상위 4비트를 다음과 같이 채웁니다.
 - PC+4(=0x10000000)에서 0x1을 가져온다.
 - PC(=0x0FFFFFFC)에서 0x0을 가져온다.
- 실제로는 어떻게 작동되는지 알아보기 위한 테스트용 MIPS Assembly를 아래와 같이 작성했습니다.

```

.text
main:
    # 1. 목적지 주소에 명령어 저장
    li $t1, 0x24110002      # li $s1, 2
    li $t0, 0x00000400
    sw $t1, 0($t0)         # 0x00000400에 저장

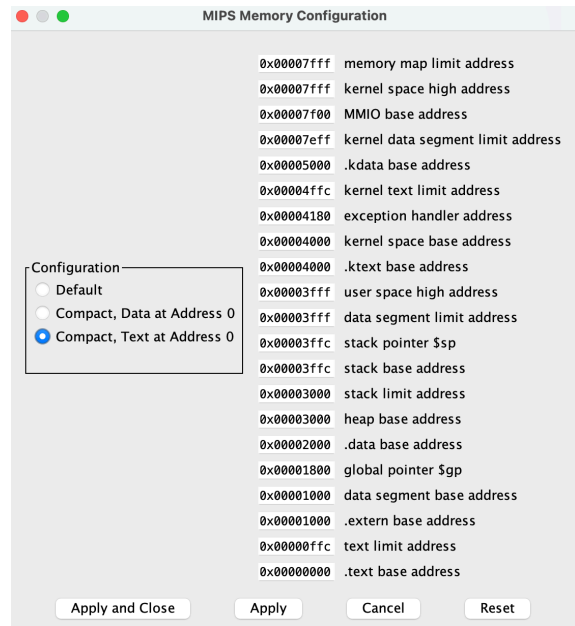
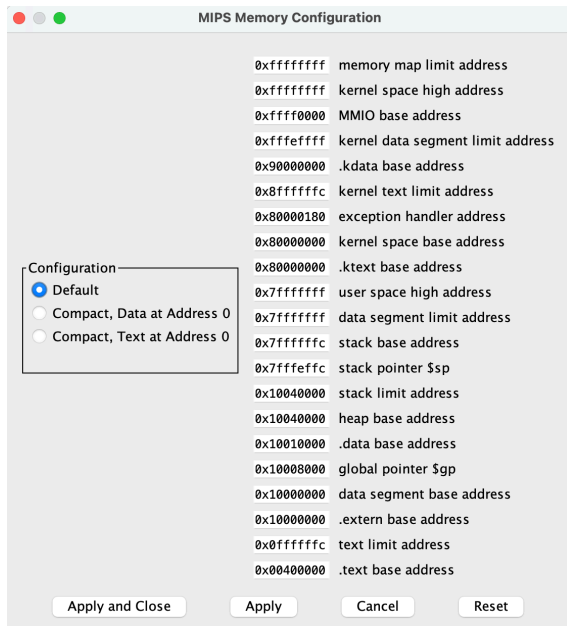
    li $t1, 0x24110001      # li $s1, 1
    lui $t0, 0x1000
    ori $t0, $t0, 0x0400
    sw $t1, 0($t0)         # 0x10000400에 저장

```

```
# 2. 임계지점에 Jump 명령어 저장
li $t1, 0x08000100      # j 0x100 (Offset 0x100)
lui $t0, 0x0FFF
ori $t0, $t0, 0xFFFC
sw $t1, 0($t0)          # 0x0FFFFFFC에 저장

# 3. 임계지점으로 바로 이동
jr $t0
```

- Mars에서 해당 어셈블리를 실행한 결과입니다.



- 메모리의 주소 범위를 벗어났다는 오류가 발생하며 실행이 중단되었습니다. 이에 Memory Configuration 탭에 들어가보니 메모리의 각 영역별로 할당된 범위가 있었고, default configuration만 간단히 살펴보았을 때 .text 영역의 제한이 우리가 생각했던 임계지점과 동일했습니다.



- 이를 고려하여 오프셋을 조금 수정해본 결과 .text 영역에는 쓰기 권한이 없다는 오류와 함께 종료되었습니다.
- 얇은 컴퓨터 지식으로 인해 더이상의 사고를 중단하고 어셈블리를 이용한 테스트를 중단했습니다.

- 여기까지의 중간 결론은 “PC를 사용하는지 PC+4를 사용하는지는 제대로 명시만 해주면 되고, 이후 메모리 관리나 명령어 대치 등 추가로 필요한 부분은 운영체제나 컴파일러 등 외부적인 요소를 이용해 해결한다” 입니다. 특히, 메모리의 크기가 작다면, 상위 4비트라는 아주 높은 영역은 무관심한 영역 혹은 상관이 없는 영역이라고 보아도 무방합니다.

-
- 결국 중요한 점은, 우리가 구현한 CPU에서 해당 문제로 인한 오류의 발생 여부입니다.
 - 우리가 Verilog 구현한 CPU와 메모리의 코드를 분석하는 방향으로 바뀌어왔습니다.

MEM.v

```
module MEM(
    ...

    reg [31:0] memory [0:8191];

    ...
endmodule
```

- 우리의 메모리 용량은 $4\text{byte} * 8192 = 32\text{KiB}$ 로, 이론상 32bit CPU가 가질 수 있는 최대 메모리 크기인 4GiB에 한참 못미치는 수준입니다.
- 상위 4비트의 자리올림이 원인이 되는 이 문제가 발생하기 위해서는 메모리가 256MiB보다 커야 하므로 우리의 자그마한 메모리에서는 PC를 사용해도, PC+4를 사용해도 같은 연산 결과가 도출될 것입니다.